第 2 章：HTTP 協定

(HyperText Transfer Protocol)

HTTP 是網際網路的基石，定義了客戶端與伺服器之間交換資料的規則。

2.1 HTTP 基本概念

- HTTP 是一種**基於文本**的協議。
- 運作於傳輸層的 **TCP** 之上（HTTP/3 則轉向 UDP/QUIC）。

TCP 的核心特色

1. **可靠性 (Reliability)**：確保數據完整。
2. **三向交握 (Three-way Handshake)**：確認收發能力。
3. **有序傳輸 (Ordered Delivery)**：保證順序。
4. **流量控制 (Flow Control)**：防止溢位。

2.1.2 HTTP 發展歷史 ⏳

- **HTTP/0.9 (1991)**：Tim Berners-Lee 發明，跨平台分享文件。
- **HTTP/1.0 (1996)**：新增標頭 (Headers)、狀態碼。
- **HTTP/1.1 (1997)**：**持久連接 (Keep-Alive)**、快取管理（最普及）。
- **HTTP/2 (2015)**：效能大躍進。**多路復用 (Multiplexing)**。
- **HTTP/3 (2022)**：基於 UDP 的 **QUIC**，減少延遲，網路切換穩定。

Keep-Alive 機制

- **HTTP/1.0**：每請求一個檔案就建立一次 TCP 連線，完畢即斷開。
- **HTTP/1.1 (預設啟用)**：**持久連接 (Persistent Connection)**。
- **好處**：
  - 多個請求/回應共用同一個 TCP 連線。
  - 大幅減少重複「三次握手」的延遲。

2.1.3 HTTP 方法 (HTTP Methods)

方法	描述
GET	請求資源，通常用來讀取資料。
POST	提交資料，通常用來創建或修改。
PUT	更新資料。
DELETE	刪除資料。
HEAD	僅返回標頭 (Headers)。
PATCH	部分修改資料。

2.1.4 HTTP 請求的生命週期

1. **DNS 查詢**: 域名 -> IP。
2. **TCP 連接**: 三次握手。
3. **TLS 握手**: 密鑰交換（HTTPS）。
4. **傳送請求**: 客戶端發送 Data。
5. **伺服器處理**: 程式碼運算 & 資料庫查詢。
6. **傳送回應**: 伺服器回傳 Data。
7. **瀏覽器渲染**: 解析 HTML/CSS/JS。

2.1.5 HTTPS 與傳輸安全

- **HTTP**: 不加密，易被竊聽。
- **HTTPS**: 使用 **TLS/SSL** 加密。

核心機制

1. **數位憑證 (Digital Certificate)**：伺服器身分證 (CA 簽發)，包含公鑰。
2. **確保隱私 (Privacy)**：透過加密（對稱/非對稱）轉換為亂碼。
3. **確保完整性 (Integrity)**：雜湊 (Hashing) 與數位簽章，防止篡改。

2.2 HTTP 請求與回應結構

網址 (URL) 拆解

```
https://www.example.com:443/person/123?name=John&age=25
```

- **https**：通訊協定 (Protocol)。
- **www.example.com**：網域位址 (Host)。
- **:443**：連接埠 (Port)。
- **/person/123**：資源路徑 (Path)。
- **?name=John&age=25**：查詢參數 (Query Parameters)。

2.2.1 HTTP 請求 (Request) 範例

```
GET /course/ch02_http.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Macintosh)
Accept: text/html
```

- **GET**: 方法。
- **/course/ch02_http.html**: 路徑。
- **HTTP/1.1**: 協定版本。
- **Host**: 目標域名。

2.2.2 HTTP 回應 (Response) 範例

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Connection: keep-alive

<!DOCTYPE html>...
```

- **200 OK**: 狀態碼與訊息。
- **Content-Type**: 資料性質（HTML/圖片...）。
- **Content-Length**: 回應正文大小。

2.2.3 HTTP 狀態碼 (Status Codes)

- **1xx (Informational)**: 處理中。
- **2xx (Success)**: 成功 (200 OK)。
- **3xx (Redirection)**: 重新導向 (301 Moved Permanently)。
- **4xx (Client Error)**: 客戶端錯誤 (404 Not Found)。
- **5xx (Server Error)**: 伺服器錯誤 (500 Internal Error)。

2.3 實作演練：DevTools 觀察員

常用面板

- **Elements**: HTML 結構與 CSS 樣式。
- **Console**: JavaScript 偵錯與 Log。
- **Network**: **核心工具**。記錄所有網路傳輸。

Network 觀察重點

- **Headers**: General, Response, Request 標頭。
- **Payload**: POST 提交的資料。
- **Preview/Response**: 原始回傳內容。

2.3.1 Network Timing (Waterfall)

五大階段

1. **Queueing (排隊)**: 等待連線。
2. **Stalled (停滯)**: 等待 Socket。
3. **DNS/TCP/SSL**: 連線建立消耗。
4. **TTFB (Time to First Byte)**: **效能關鍵**。伺服器處理時間。
5. **Content Download**: 資料接收時間。

2.3.2 終端機大師 (curl)

OS 指導

- Windows: `cmd` / `powershell` (內建)。
- macOS: `Terminal` (Spotlight 搜尋)。
- Linux: `Ctrl + Alt + T` 。

練習指令

- 僅看標頭： `curl -I https://www.fcu.edu.tw`
- 詳細模式： `curl -v https://www.fcu.edu.tw` (包含 TCP 交握)

Q & A

感謝大家！準備好開始分析封包了嗎？